WATER QUALITY PREDICTION SYSTEM

Python + Machine Learning + Streamlit

This PDF contains the main Python code for the software-based micro project: a Random Forest model for classifying labelled water-quality data and a browser-based interface for entering parameter values.

Project files included

• train_model.py — loads the dataset, preprocesses data, trains and evaluates the model.

• app.py — opens the Streamlit interface and displays predictions.

• requirements.txt — lists the Python packages required.

Important: The prediction is only an estimate based on the dataset. It does not prove that water is safe to drink; appropriate laboratory testing and applicable drinking-water standards are still required.

1. Folder Structure

```
water_quality_project/
■■■ water_potability.csv
■■■ train_model.py
■■■ app.py
■■■ requirements.txt
■■■ water_quality_model.joblib  (created after training)
```

Use a labelled CSV dataset named **water_potability.csv**. This code expects a target column named **Potability** and numeric input columns. If your dataset uses a different target name, update TARGET_COLUMN in train_model.py. The feature columns used by the web form are taken from the actual dataset.

2. requirements.txt

```
pandas
scikit-learn
joblib
streamlit
```

3. Model Training Code — train_model.py

Save the following code in a file named **train_model.py**.

```
import pandas as pd
import joblib

from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix
)

DATA_PATH = "water_potability.csv"
TARGET_COLUMN = "Potability"
MODEL_PATH = "water_quality_model.joblib"


def main():
    # Load the dataset
    df = pd.read_csv(DATA_PATH)

    if TARGET_COLUMN not in df.columns:
        raise ValueError(
            f"Target column '{TARGET_COLUMN}' was not found. "
            f"Available columns: {list(df.columns)}"
        )

    # Remove rows without a target label
    df = df.dropna(subset=[TARGET_COLUMN]).copy()

    # Use numeric columns as model inputs
    X = df.drop(columns=[TARGET_COLUMN])
    X = X.select_dtypes(include="number")
    y = df[TARGET_COLUMN]

    if X.empty:
        raise ValueError("No numeric input features were found.")

    if y.nunique() < 2:
        raise ValueError("The target must contain at least two classes.")

    stratify_labels = y if y.value_counts().min() >= 2 else None

    X_train, X_test, y_train, y_test = train_test_split(
        X,
        y,
        test_size=0.20,
        random_state=42,
        stratify=stratify_labels
    )

    # Fill missing numeric values and train the classifier
    model = Pipeline(steps=[
        ("imputer", SimpleImputer(strategy="median")),
        ("classifier", RandomForestClassifier(
            n_estimators=200,
            random_state=42,
            class_weight="balanced"
```

```
        ))
    ])

    model.fit(X_train, y_train)
    predictions = model.predict(X_test)

    print("Number of dataset rows:", len(df))
    print("Input features:", list(X.columns))
    print("Test accuracy:", round(accuracy_score(y_test, predictions), 4))
    print("\\nClassification report:\\n",
          classification_report(y_test, predictions, zero_division=0))
    print("Confusion matrix:\\n", confusion_matrix(y_test, predictions))

    # Save the model and exact feature order for the web application
    bundle = {
        "model": model,
        "features": list(X.columns),
        "target_column": TARGET_COLUMN
    }
    joblib.dump(bundle, MODEL_PATH)
    print(f"Saved trained model to {MODEL_PATH}")


if __name__ == "__main__":
    main()
```

4. Web Application Code — app.py

Save the following code in a file named **app.py**. It creates a form automatically from the numeric feature columns used during model training.

```
import joblib
import pandas as pd
import streamlit as st

MODEL_PATH = "water_quality_model.joblib"

st.set_page_config(
    page_title="Water Quality Prediction",
    page_icon="■",
    layout="centered"
)

st.title("■ Water Quality Prediction System")
st.write(
    "Enter the measured water parameters. The result is a machine-learning "
    "prediction based on the selected dataset, not a drinking-water "
    "safety certificate."
)


@st.cache_resource
def load_bundle():
    return joblib.load(MODEL_PATH)


try:
    bundle = load_bundle()
except FileNotFoundError:
    st.error("Model file not found. Run `python train_model.py` first.")
    st.stop()

model = bundle["model"]
features = bundle["features"]

st.subheader("Water Quality Parameters")

with st.form("water_quality_form"):
    values = {}
    columns = st.columns(2)

    for i, feature in enumerate(features):
        with columns[i % 2]:
            values[feature] = st.number_input(
                feature,
                value=0.0,
                format="%.4f",
                help="Enter a measured value using the units expected by "
                     "your dataset."
            )

    submitted = st.form_submit_button("Predict Water Quality")

if submitted:
    input_df = pd.DataFrame([values], columns=features)
    prediction = model.predict(input_df)[0]

    st.subheader("Prediction Result")
    st.write(f"Predicted dataset class: **{prediction}**")
    st.warning(
        "This result is only a model estimate. It does not confirm that "
        "water is safe to drink. Use suitable laboratory testing and "
        "applicable drinking-water standards for safety decisions."
    )
```

5. How to Run the Project

1. Install Python 3.10 or later and open the project folder in VS Code.

2. Place your labelled dataset in the folder and name it water_potability.csv.

3. Open the VS Code terminal in that folder.

4. Install the required libraries: pip install -r requirements.txt

5. Train and evaluate the model: python train_model.py

6. Start the web application: streamlit run app.py

7. Open the local browser page shown in the terminal, enter measured parameter values, and click Predict Water Quality.

6. Notes

• The model file water_quality_model.joblib is generated when train_model.py finishes successfully.

• The accuracy printed by the program depends on the dataset and must be measured by running the code; do not assume or invent an accuracy value.

• The parameter units and target-label meanings must match the dataset documentation.

• Do not use this educational application as the only basis for deciding whether water is safe to drink.